



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI
INFORMATICA - SCIENZA E INGEGNERIA

STRUTTURE

COSIMO LANEVE

`cosimo.laneve@unibo.it`

CORSO 00819 – PROGRAMMAZIONE

ARGOMENTI (SEZIONE 10.1 SAVITCH)

1. dichiarazioni di strutture
2. l'operazione di selezione e i campi
3. strutture passate come parametri e ritornate come valori di funzioni
4. esempi/esercizi

PROLOGO

una **struttura** è un dato composto da elementi **in generale differenti** tra loro (eterogenei), raggruppati sotto un unico nome

* le strutture sono anche chiamati **record**

* anche gli array sono dei raggruppamenti di dati, **ma dello stesso tipo**

esempi: giorno 19 12 2001
 day month year

studente *Marco Bianchi* *Unibo* 13 10 1975
 nome cognome istituto day month year
 data_di_nascita

volo aereo AZ506 B747 255 
 n_volo tipo_areo n_posti nome_passeggeri
 (array)

STRUTTURE/ESEMPI

in C++ è possibile definire tipi di dato che raggruppano in una unica struttura informazioni di tipo diverso

occorre specificare:

- * il nome di ciascuna componente (i componenti sono detti **campi**)
- * il tipo di informazione memorizzato nelle componenti

esempi:

```
struct data {  
    int day ;  
    int month ;  
    int year ;  
};
```

← è il nome del
tipo struttura

ricordatevi di
questo ;

```
struct volo_aereo {  
    char n_volo [5] ;  
    char tipo_aereo [5] ;  
    int n_posti ;  
    struct passeggero nome_passeggero[255];  
};
```

```
struct studente {  
    char nome [10] ;  
    char cognome [10] ;  
    char istituto [10] ;  
    struct data data_nasc ;  
};
```

↑ una struttura che
contiene un'altra struttura

STRUTTURE/DICHIARAZIONE

sintassi: `struct id {type1 id_list1 ; type2 id_list2 ; ... ; typen id_listn ;} ;`

- * l'identificatore `id` è il nome del tipo struttura
- * ogni `id_listi` è un elenco di uno o più identificatori (nomi di campi) separati da virgole;
- * il tipo di ogni campo in `id_listi` è `typei` (`typei` è un tipo semplice o un tipo precedentemente definito)

osservazione: la dichiarazione di struttura serve a quantificare lo spazio di memoria necessario per le variabili di quel tipo

una volta definita una struttura, è possibile dichiarare variabili di quel tipo

```
data oggi, giorno_fortunato ;  
studente Pinco_Pallino, Marco_Rossi ;  
volo_aereo volo_Parigi ;
```

STRUTTURE/OPERAZIONE DI SELEZIONE

selezione: operazione per accedere ai campi (*dot-notation*)

sintassi: $lhs_expression ::= \begin{array}{l} . \quad . \quad . \\ | \quad lhs_expression \ . \ identifier \end{array}$

esempi: `volo_Parigi.n_posti = 250 ;`
`oggi.day = oggi.day + 2 ;`

STRUTTURE/OPERAZIONE DI SELEZIONE

esempio:

```
Pinco_Pallino.nome[0] = 'P' ;
```

è un array

selezione su array

NOTATE la differenza tra l'operazione di selezione nelle strutture e quella negli array

questa differenza consente di evitare errori di tipo

* in un array **A** con due elementi è sempre possibile fare **A[0] = A[1] ;**

perchè **A[0]** e **A[1]** hanno lo stesso tipo

* in una struttura **A** con due campi "zero" e "uno", l'assegnamento

A.zero = A.uno ;

richiede che i due campi abbiano lo stesso tipo (**ed in generale non è così**)

STRUTTURE/OPERAZIONE DI COPIA

è possibile copiare e assegnare strutture

```
Marco_Rossi = Pinco_Pallino ;
```

risultato: tutti i campi di `Pinco_Pallino` sono copiati nei corrispondenti campi di `Marco_Rossi`

* anche quando i campi sono array (vedi `nome`, `cognome`, `istituto`)

* in ciò le strutture si differenziano dagli array

vincolo: non è possibile confrontare strutture

STRUTTURE/INIZIALIZZAZIONE

è possibile inizializzare una struttura quando viene dichiarata

```
struct date {  
    int month ;  
    int day ;  
    int year ;  
} ;
```

è possibile dichiarare ed inizializzare una variabile in questo modo:

```
date today = { 10, 24, 2014 } ;
```

risultato: i campi month, day e year di today sono
inizializzati a 10, 24, 2014 rispettivamente

STRUTTURE COME PARAMETRI DI FUNZIONI

le strutture **possono essere passati come argomenti di funzioni**
(come gli array) **ed anche essere restituite come valori**

esempio:

```
struct point { int x ; int y ; } ;
```

potete anche scrivere
struct point

```
point make_point(int a, int b){  
    point tmp ;  
    tmp.x = a ; tmp.y = b ;  
    return(tmp) ;  
}
```

problema: cosa accade se si definisce

```
void incr_point (point a){  
    a.x = a.x + 1 ;  
    a.y = a.y + 1 ;  
}
```

e nel main si invoca

```
. . .    point b ;  
          b.x = 1 ; b.y = 1 ;  
          incr_point(b) ; . . .
```

STRUTTURE COME PARAMETRI DI FUNZIONI/CONT.

risposta: a differenza degli array, il passaggio delle strutture come parametro è **per valore**, **compreso il caso in cui un campo della struttura è un array** (viene copiato)

è possibile passare una struttura per riferimento:

```
void incr_point (point& a){  
    a.x = a.x + 1 ;  
    a.y = a.y + 1 ;  
}
```

se nel main si invoca

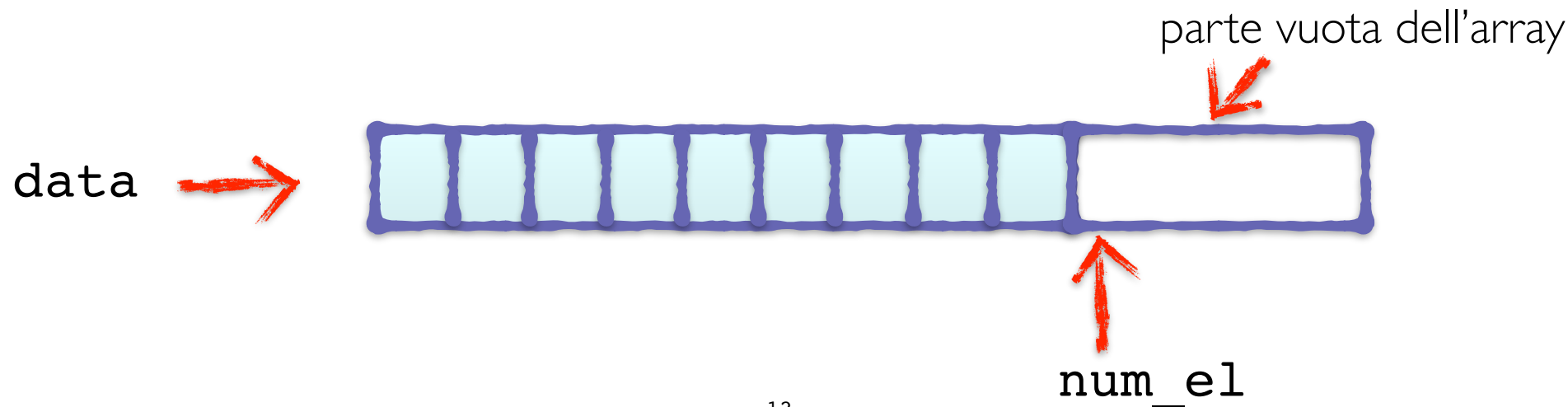
```
    . . .  
    point b ;  
    b.x = 1 ; b.y = 1 ;  
    incr_point(b) ;  
    . . .
```

allora l'incremento dei due campi è visibile nel chiamante...

ESERCIZIO: TIPO DI DATO INSIEME

esercizio: implementare il tipo di dato insieme di interi (per mezzo di strutture e array)

- * l'implementazione deve fornire le seguenti operazioni: **creazione** di un insieme vuoto, **appartenenza** di un elemento all'insieme, **intersezione** e **unione**
- * implementiamo un insieme attraverso una struttura con due campi: un campo **data** che è un array e un campo **num_el** che contiene il numero di elementi nell'array



ESERCIZIO: TIPO DI DATO INSIEME

```
const int length = 1000;

struct set {
    int data[length];
    int num_el;
} ;

set empty_set() {
    set tmp ;
    tmp.num_el = 0 ;
    return(tmp) ;
}
```

non esiste un elemento
in data[-1]: se num_el == 0
significa che l'insieme è vuoto

STRUTTURE/TIPO DI DATO INSIEME/CONT.

gli elementi nell'array sono ordinati

le operazioni saranno più efficienti

```
bool is_in(set s, int e){  
    if (s.num_el == 0) return(false) ;  
    else {  
        int l = 0 ;  
        int r = s.num_el ;  
        int m ;  
        bool found = false ;  
        while ((l <= r) && !found) {  
            m = (l+r)/2 ;  
            if (s.data[m] == e) found = true ;  
            else if (s.data[m] > e) r = m-1 ;  
            else l = m+1 ;  
        }  
        return(found) ;  
    }  
}
```

algoritmo di ricerca binaria
perché l'array è ordinato

STRUTTURE/TIPO DI DATO INSIEME/CONT.

```
set intersection(set s1, set s2){
    if ((s1.num_el == 0) || (s2.num_el == 0))
        return(empty_set()) ;
    else {
        set s = empty_set() ;
        int i = 0 ;
        int j = 0 ;
        while ((i < s1.num_el) && (j < s2.num_elem)) {
            if (s1.data[i] == s2.data[j]) {
                s.data[s.num_el] = s1.data[i] ;
                s.num_el = s.num_el + 1 ;
                i = i+1 ;
                j = j+1 ;
            } else if (s1.data[i] < s2.data[j]) i = i+1 ;
            else j = j+1 ;
        }
        return(s) ;
    }
}
```

trascrizione di due array ordinati
in un terzo array (ordinato)

STRUTTURE/TIPO DI DATO INSIEME/CONT.

```
set set_union(set s1, set s2){
    if (s1.num_el == 0) return(s2) ;
    else if (s2.num_el == 0) return(s1) ;
    else { set s = empty_set() ;
        int i1 = 0 ;
        int i2 = 0 ;
```

unione di due array ordinati

```
while ((i1 < s1.num_el) && (i2 < s2.num_el) && (s.num_el < length)){
    if (s1.data[i1] < s2.data[i2]) {
        s.data[s.num_el] = s1.data[i1] ;
        s.num_el = s.num_el + 1 ;
        i1 = i1+1 ;
    } else if (s1.data[i1] > s2.data[i2]){
        s.data[s.num_el] = s2.data[i2] ;
        s.num_el = s.num_el + 1 ;
        i2 = i2+1 ;
    } else {
        s.data[s.num_el] = s1.data[i1] ; s.num_el = s.num_el + 1 ;
        i1 = i1+1 ; i2 = i2+1 ;
    }
}
```

```
while ((i1 < s1.num_el) && (s.num_el < length)){
    s.data[s.num_el] = s1.data[i1] ; s.num_el = s.num_el + 1 ; i1 = i1+1 ;
}
while ((i2 < s2.num_el) && (s.num_el < length)){
    s.data[s.num_el] = s2.data[i2] ; s.num_el = s.num_el + 1 ; i2 = i2+1 ;
}
if ((i1 < s1.num_el) || (i2 == s2.num_el)) cout << "write error" << endl ;
return(s) ;
```

```
}
```

```
}
```

copia
delle parti
restanti

STRUTTURE/ESERCIZI [27/5/22]

1. Un nodo è un elemento il cui tipo è `struct nodo { int val; int next ; }`. Una lista di nodi è implementata attraverso un array di elementi `nodo` la cui testa si trova ad indice 0 e il cui campo `next` indica la posizione dell'elemento successivo. Se un nodo ha `next == -1` allora quel nodo è finale (non c'è alcun elemento successivo). Ad esempio

1; 4		3; -1		5; 2
0	1	2	3	4

implementa la lista 1.5.3. Si osservi che i nodi **non** sono memorizzati in maniera contigua.

(a) scrivere una funzione

```
void stampa (nodo A[], const int length)
```

che prende una lista di nodi `A` e stampa i campi `val` dei nodi.

(b) scrivere una funzione

```
int get_last (nodo A[], const int length)
```

che prende una lista di nodi `A` e ritorna il campo `val` memorizzato nell'ultimo nodo.

STRUTTURE/ESERCIZI/CONT.

(c) Definire una funzione

```
void revert (nodo A[], const int length)
```

che prende una lista di nodi **A** e modifica **A** in maniera tale che rappresenti la lista invertita.

[Attenzione al vincolo che l'elemento ad indice 0 di **A** deve rappresentare la testa della lista.]

(d) Definire una funzione

```
void remove_last(nodo A[], const int length)
```

che prende una lista di nodi **A** e modifica **A** in maniera da eliminare l'ultimo elementop in **A**. La funzione non fa nulla se **A** contiene solamente l'elemento in posizione 0.

ESERCIZIO: IL TIPO DI DATO PILA

implementare il tipo di dato pila di interi (LIFO) (per mezzo di strutture e array)

* l'implementazione deve fornire le operazioni: creazione di una pila vuota, test per pila vuota, inserimento e estrazione

```
const int length = 1000;

struct stack { int data[length];
               int num_el;
} ;

stack new_stack() {
    stack tmp;
    tmp.num_el = 0;
    return(tmp) ;
}

bool is_empty(stack s) {
    return(s.num_el == 0) ;
}
```

non esiste un elemento in
data[length]: se num_el==0
significa che la pila è vuota
se num_el==length
significa che la pila è piena

STRUTTURE/TIPO DI DATO PILA/CONT

```
stack push(stack s, int e){
    if (s.num_el == length){
        cout << "error, stack full\n" ;
    } else { s.data[s.num_el] = e ; s.num_el = s.num_el + 1;}
    return(s) ;
}

stack pop(stack s){
    if (s.num_el == 0) { cout << "error, empty stack\n"; }
    else { cout << "pop" << s.data[s.num_el-1] << "\n" ;
           s.num_el = s.num_el - 1; }
    return(s) ;
}
```

osservazione: la complessità computazionale di ogni operazione è costante

problemi: (1) **le risposte erranee "a video" non sono ideali** -- *non ci dovrebbero essere errori!*

(2) non c'è incapsulamento: la struttura degli elementi di tipo stack è visibile all'utente, che può modificarli senza usare le funzioni implementate